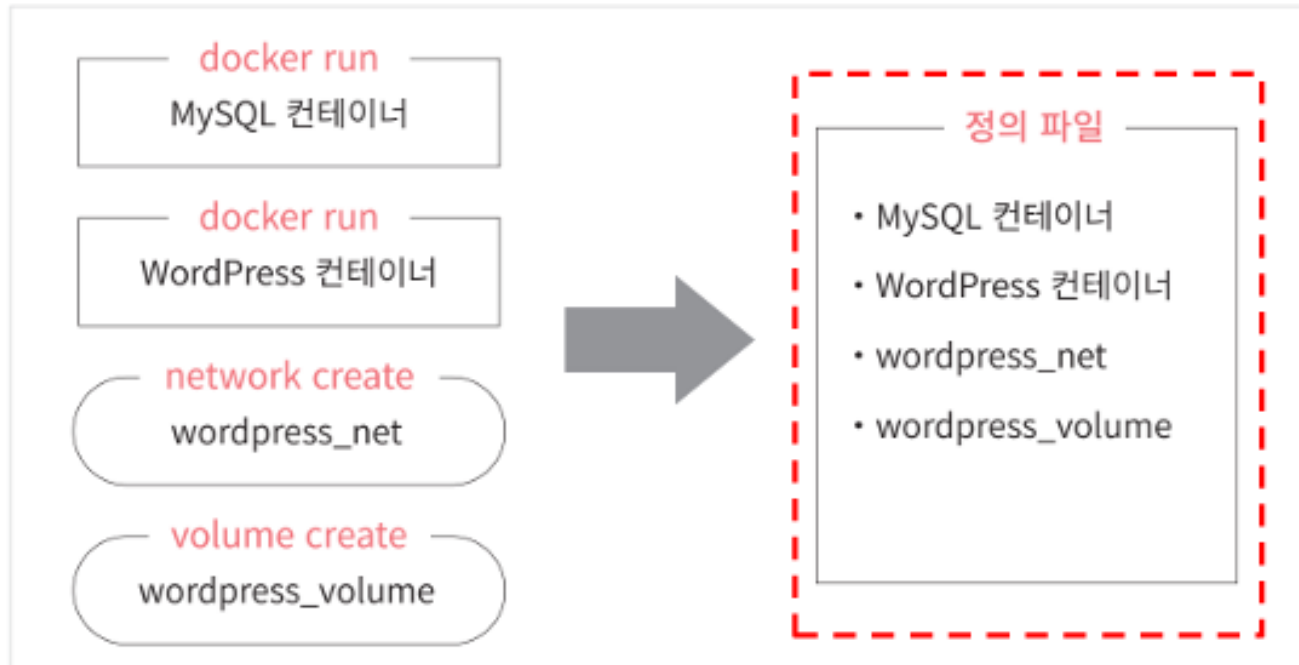


도커 컴포즈

1절 - 도커 컴포즈란?

도커 컴포즈

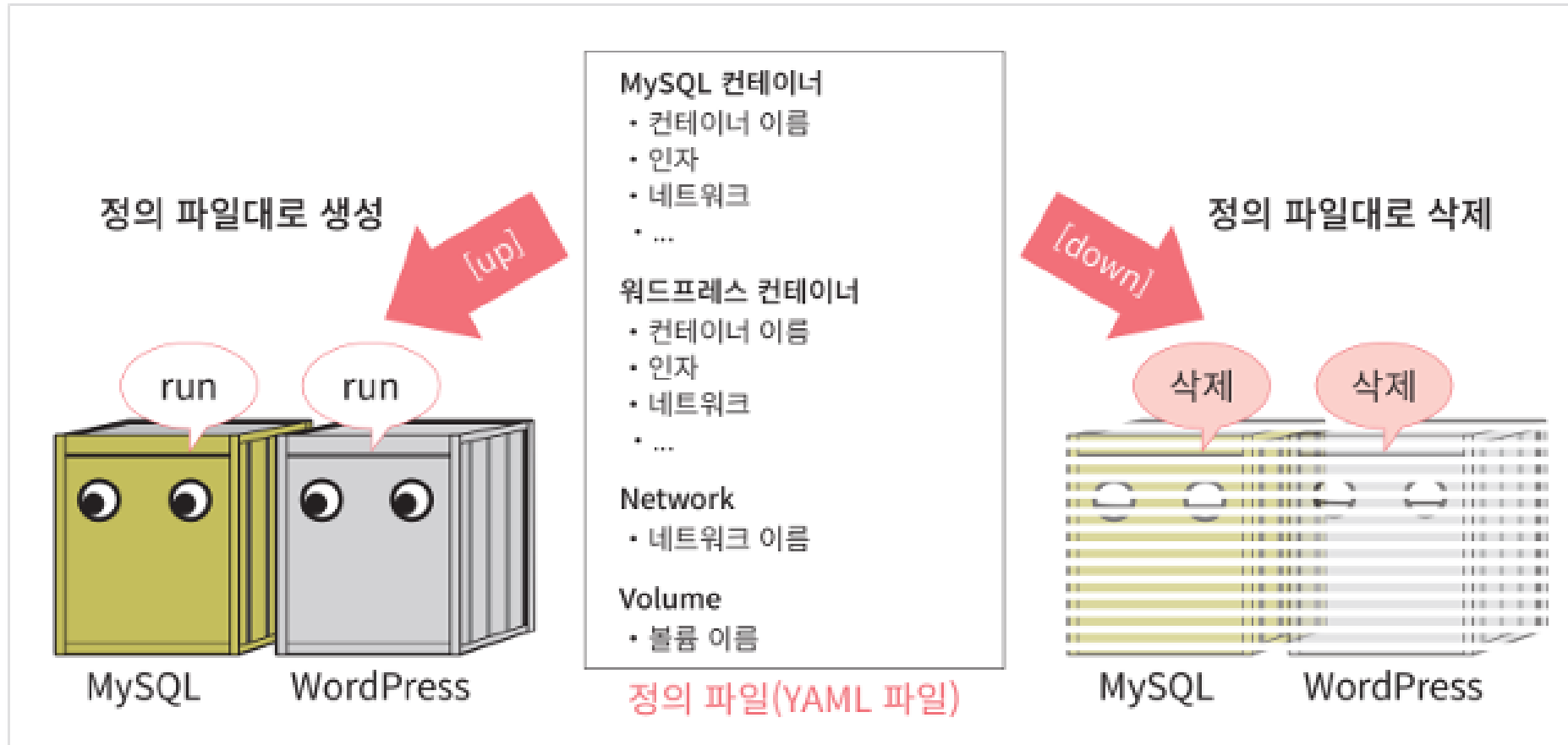
- 시스템 관련 명령들을 하나의 파일에 작성하여 한번에 시스템 전체를 실행하고 종료 및 폐기까지 실행하도록 도와주는 도구
- 도커 컴포즈를 사용하면 여러 개의 명령어를 하나의 정의 파일로 합쳐 실행할 수 있음



도커 컴포즈

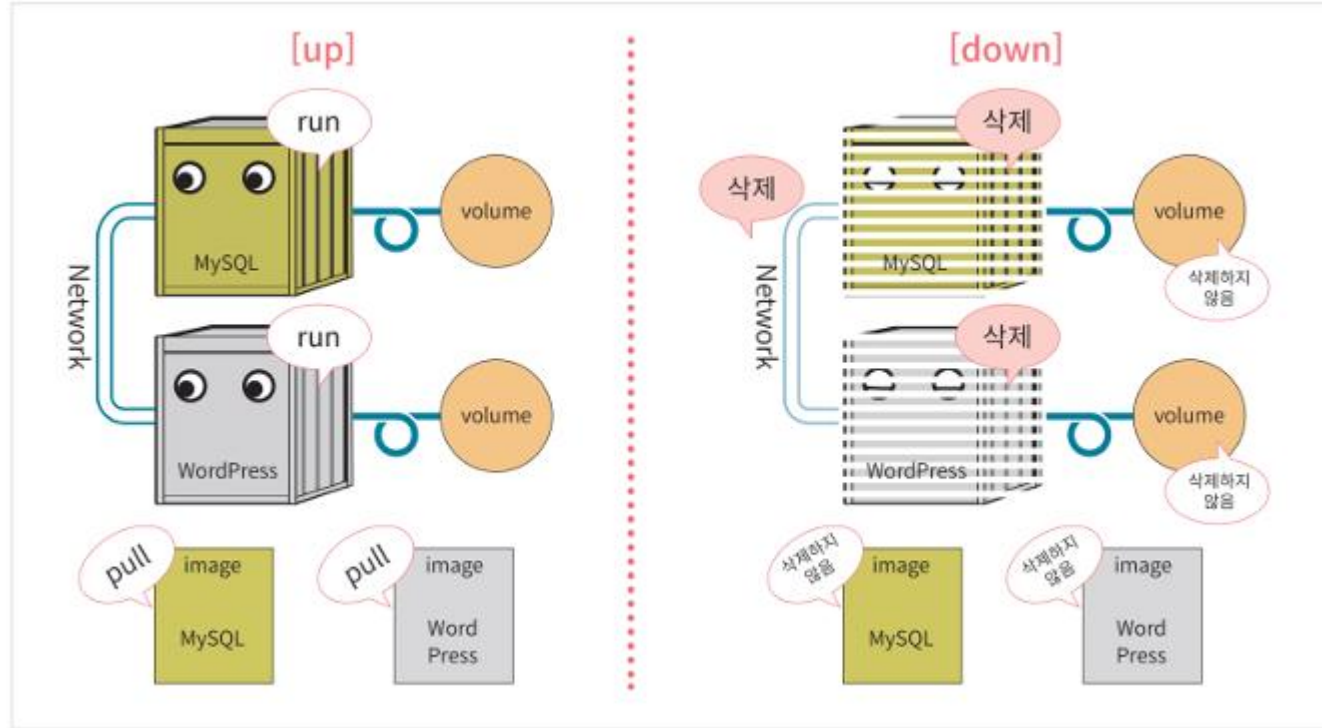
● 도커 컴포즈 구조

- YAML(YAML Ain't a Markup Language)포맷의 정의 파일
- 도커 컴포즈는 시스템의 모든 정보를 정의 파일에 기재함



도커 컴포즈

- up 커맨드로 생성 및 시작, down 커맨드로 정지 및 삭제

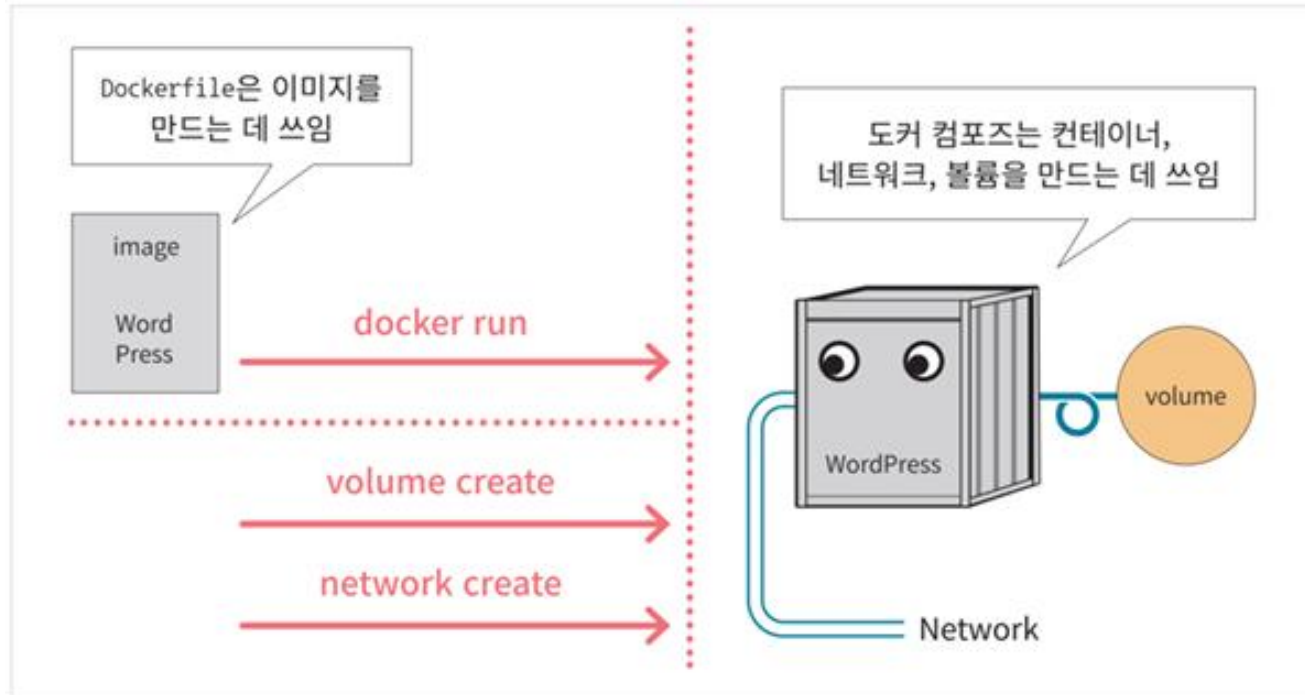


- up 명령 : 정의 파일의 내용대로 이미지를 내려 받고 컨테이너를 생성 및 실행, 네트워크나 볼륨도 정의 가능
- down 명령 : 컨테이너와 네트워크를 정지 및 삭제
- Stop 명령 : 컨테이너와 네트워크를 삭제없이 종료만 진행

도커 컴포즈

● 도커 컴포즈와 Dockerfile 스크립트 차이점

- 도커 컴포즈 : 컨테이너와 주변 환경 및 네트워크와 볼륨까지 만들 수 있다.
- Dockerfile 스크립트 : 이미지를 만들기 위한 것으로 네트워크나 볼륨은 만들 수 없다.



<COLUMN> 도커 컴포즈와 쿠버네티스의 차이점

쿠버네티스 : 도커 컨테이너 관리 도구

도커 컴포즈 : 컨테이너 생성 및 삭제, 관리 기능은 없음

도커 컴포즈

2절 - 도커 컴포즈 설치와 사용법

도커 컴포즈 설치

- <실습> 도커 컴포즈 설치

- 도커 컴포즈

- 도커 엔진과 별개의 소프트웨어, 사용법은 유사
- 파이썬으로 작성된 프로그램

- 도커 데스크톱에는 도커 컴포즈 함께 설치되어 있음
- 우분투 리눅스에서 도커 컴포즈 설치

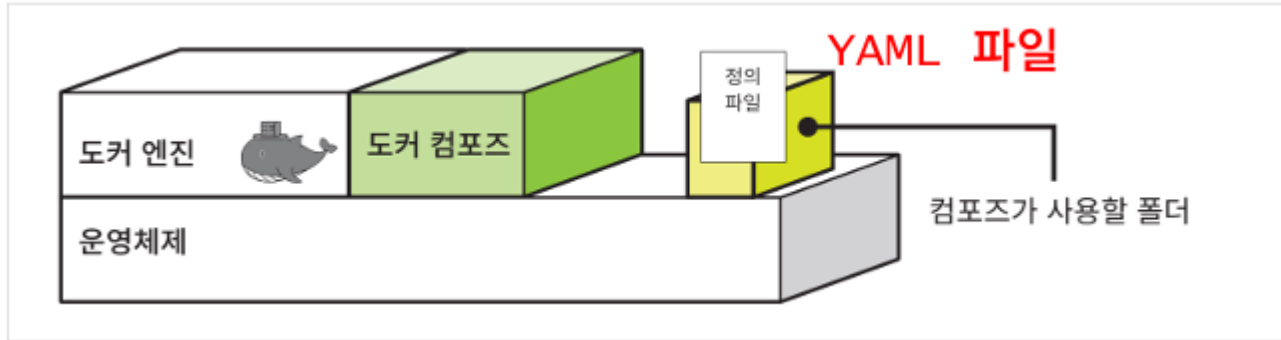
```
sudo apt install -y python3 python3-pip
sudo pip3 install docker-compose
```

- 확인

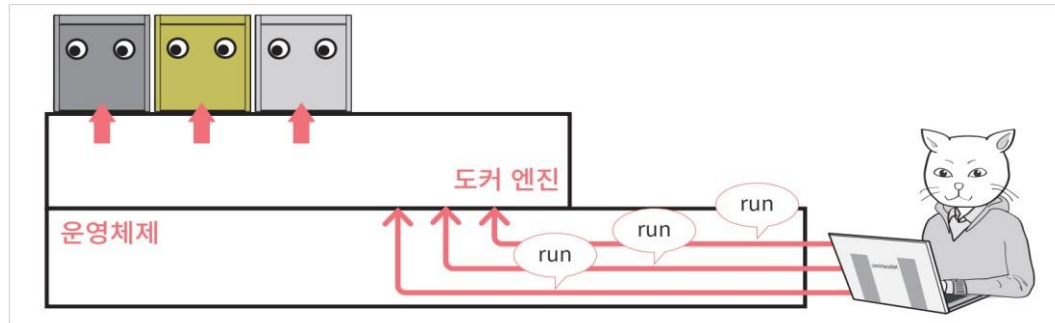
```
docker-compose --version
```

도커 컴포즈 설치

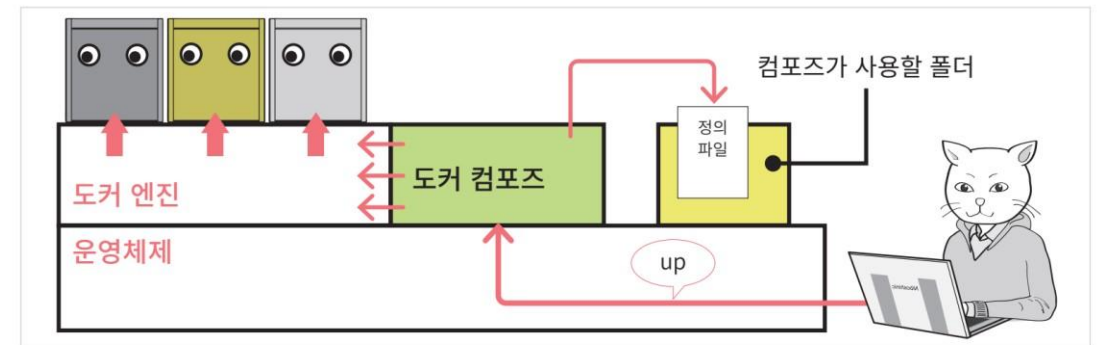
- 도커 컴포즈 사용법 - 호스트 컴퓨터에 정의 파일을 준비한다.



- 정의 파일명 : docker-compose.yml
- 도커 엔진에 의해 수행
 - 사람이 입력하는 수동 명령을 컴포즈가 대신 전달하는 개념



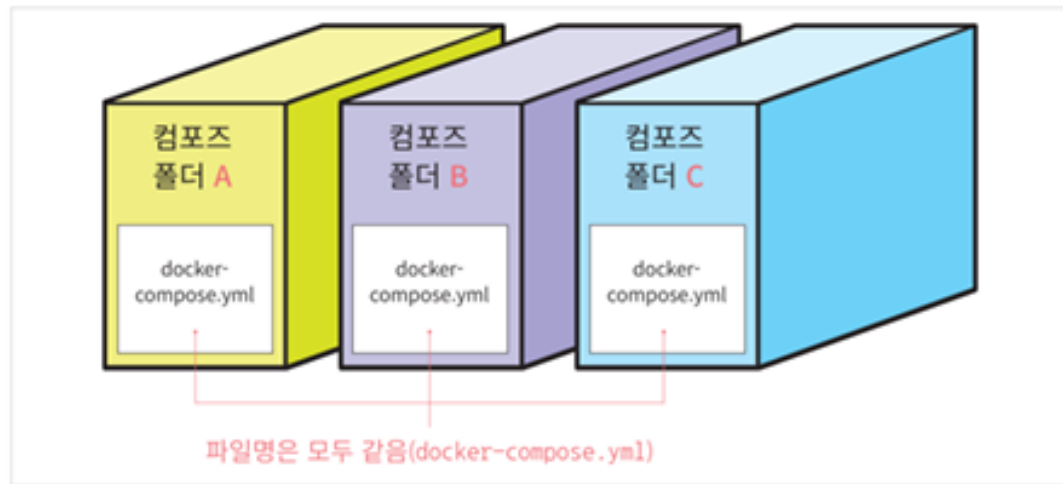
도커 컴포즈가 명령어를 대신 입력해주는 구조



도커 컴포즈 사용하기

도커 컴포즈 설치

- 정의 파일은 한 폴더에 하나만 존재해야 됨
- 컨테이너 생성에 필요한 파일(이미지,HTML등)은 같은 폴더에 존재



<COLUMN> 서비스와 컨테이너

도커 컴포즈에서 서비스란 컨테이너가 모인 것을 의미
일반적으로 컨테이너와 서비스는 동일한 개념으로 봐도 무방

도커 컴포즈

3절 - 도커 컴포즈 파일 작성 방법

도커 컴포즈 정의 파일

- 도커 컴포즈 파일의 예1)
 - 아파치 컨테이너의 컴포즈 파일 예제

```
version: "3"

services:
  apa000ex2:
    image: httpd
    ports:
      - 8080:80
    restart: always
```

```
docker run --name apa000ex2 -d -p 8080:80 httpd
```

도커 컴포즈 정의 파일

- 도커 컴포즈 파일의 예2)
 - 워드프레스 컨테이너의 컴포즈 파일 예제

```
version: "3"

services:
  wordpress000ex12:
    depends_on:
      - mysql000ex11
    image: wordpress
    networks:
      - wordpress000net1
    ports:
      - 8085:80
    restart: always
    environment:
      WORDPRESS_DB_HOST=mysql000ex11
      WORDPRESS_DB_NAME=wordpress000db
      WORDPRESS_DB_USER=wordpress000kun
      WORDPRESS_DB_PASSWORD=wkunpass
```

- wordpress000ex12 컨테이너를 실행하는 명령어

```
docker run --name wordpress000ex12 -dit
  --net=wordpress000net1
  -p 8085:80
  -e WORDPRESS_DB_HOST=mysql000ex11
  -e WORDPRESS_DB_NAME=wordpress000db
  -e WORDPRESS_DB_USER=wordpress000kun
  -e WORDPRESS_DB_PASSWORD=wkunpass
wordpress
```

도커 컴포즈 정의 파일

- 파일 형식 : YAML 형식

- 파일명의 확장자 : .yaml

항목	내용
정의 파일의 형식	YAML 형식
파일 이름	docker-compose.yaml

다른 파일명 이용시 `-f` 옵션 이용하여 지정

- 컴포즈 파일 작성 방법

- 작성 순서 : 주 항목 → 이름 추가 → 설정의 순으로 진행
- 주 항목

```
version:      # 버전 기재

services:     # 컨테이너 관련 정보

networks :    # 네트워크 관련 정보

volumes:      # 볼륨 관련 정보
```

도커 컴포즈 정의 파일

- 이름 추가 시 주 항목보다 한단 들여쓰기
- 공백으로 한단 들여쓰기 사용, 한 단의 공백 개수를 동일하게 설정
- 탭 문자는 사용 못함
- 이름 뒤에는 반드시 ':' 사용
- 한 줄에 이어서 설정이 필요한 경우 : 뒤에 공백 문자가 1개가 반드시 필요
- 이름 뒤에 여러 개의 설정이 필요한 경우 줄을 바꿔 들여쓰기와 함께 '하이픈(-)+공백 문자'와 함께 설정 내역 작성

```
version: "3"
services:
  컨테이너_이름1:
    image: 네트워크_이름
    networks:
      - 네트워크_이름1:
```

공백 두 개 →

공백 네 개 →

공백 여섯 개 →

- 들여쓰기를 일정하게 맞춘다.

도커 컴포즈 정의 파일

- docker-compose.yml 템플릿

```
version: "3"
Services:
  컨테이너_이름1:
    image: 이미지_이름
    networks:
      - 네트워크_이름
    ports:
      - 포트_설정

  컨테이너_이름2:
    image: 이미지_이름
  . . .
networks:
  네트워크_이름1:
  . . .
volumes:
  볼륨_이름1:
  볼륨_이름2:
  . . .
```

도커 컴포즈 정의 파일

● 컴포즈 파일 작성 요령

- 첫번째 줄에 도커 컴포즈 버전 작성
- 주 항목 services, networks, volumes 아래에 설정 내용 작성
- 항목 간의 상하 관계는 공백을 사용한 들여쓰기로 정함
- 들여쓰기는 같은 수의 배수만큼의 공백을 사용
- 이름은 주 항목 아래에 들여쓰기 한 다음 작성
- 컨테이너 설정 내용은 이름 아래 들여쓰기를 한 다음 작성
- 여러 항목을 작성하는 경우 줄 앞에 하이픈(-)을 붙임.
- 이름 뒤에 반드시 : 문자를 부침
- : 문자 뒤에는 반드시 공백 문자가 와야 함.
- # 은 주석 처리
- 문자열은 ‘ ‘ 또는 “ “를 이용

도커 컴포즈 정의 파일

● 컴포즈 파일의 항목(1 of 3)

주 항목

항목	내용
services	컨테이너를 정의한다.
networks	네트워크를 정의한다.
volumes	볼륨을 정의한다.

자주 나오는 정의 내용

항목	docker run 커맨드의 해당 옵션 또는 인자	내용
image	이미지 인자	사용할 이미지를 지정
networks	--net	접속할 네트워크를 지정
volumes	-v, --mount	스토리지 마운트를 설정
ports	-p	포트 설정
environment	-e	환경변수 설정
depends_on	없음	다른 서비스에 대한 의존관계를 정의
restart	없음	컨테이너 종료 시 재시작 여부를 설정

도커 컴포즈 정의 파일

- 컴포즈 파일의 항목 (2of3)

`depends_on` : 컨테이너 생성 순서나 연동 여부를 정의

예) `depends_on: -namegeuk`

현재 컨테이너를 `namegeuk` 컨테이너를 생성한 다음에 생성하라는 의미
워드프레스처럼 MySQL컨테이너가 먼저 생성되어야 하는 경우 이용

restart의 설정값

설정값	내용
no	재시작하지 않는다.
always	항상 재시작한다.
on-failure	프로세스가 0 외의 상태로 종료됐다면 재시작한다.
unless-stopped	종료 시 재시작하지 않음. 그 외에는 재시작한다.

도커 컴포즈 정의 파일

- 컴포즈 파일의 항목 (3of3)

항목	docker run 커맨드의 해당 옵션 또는 인자	내용
command	커맨드 인자	컨테이너 시작 시 기존 커맨드를 오버라이드
container_name	--name	실행할 컨테이너의 이름을 명시적으로 지정
dns	--dns	DNS 서버를 명시적으로 지정
env_file	없음	환경설정 정보를 기재한 파일을 로드
entrypoint	--entrypoint	컨테이너 시작 시 ENTRYPOINT 설정을 오버라이드
external_links	--link	외부 링크를 설정
extra_hosts	--add-host	외부 호스트의 IP 주소를 명시적으로 지정
logging	--log-driver	로그 출력 대상을 설정
network_mode	--network	네트워크 모드를 설정

도커 컴포즈 정의 파일

- <실습> 컴포즈 파일 작성



- 생성할 네트워크, 볼륨 및 컨테이너 정보

항목	값
네트워크 이름	wordpress000net1
MySQL 볼륨 이름	mysql000vol11
워드프레스 볼륨 이름	wordpress000vol12
MySQL 컨테이너 이름	mysql000ex11
워드프레스 컨테이너 이름	wordpress000ex12

도커 컴포즈 정의 파일

- 정의 내용 (1 of 2)

MySQL 컨테이너(mysql000ex11)의 정의

항목	항목 이름	값
MySQL 이미지 이름	image:	mysql:5.7
사용할 네트워크	networks:	wordpress000net1
사용할 볼륨	volumes:	mysql000vol11
마운트 위치		/var/lib/mysql
재시작 설정	restart:	always
MySQL 설정	environment:	★이 붙은 항목 설정
★MySQL 루트 비밀번호	MYSQL_ROOT_PASSWORD	myrootpass
★MySQL 데이터베이스 이름	MYSQL_DATABASE	wordpress000db
★MySQL 사용자 이름	MYSQL_USER	wordpress000kun
★MySQL 비밀번호	MYSQL_PASSWORD	wkunpass

도커 컴포즈 정의 파일

- 정의 내용 (2of2)

워드프레스 컨테이너(wordpress000ex12)의 정의

항목	항목 이름	값
의존관계	depends_on:	mysql000ex11
워드프레스 이미지 이름	image:	wordpress
사용할 네트워크	networks:	wordpress000net1
사용할 볼륨	volumes:	wordpress000vol12
마운트 위치		/var/www/html
포트 번호 설정	port:	8085:80
재시작 설정	restart:	always
데이터베이스 관련 정보	environment:	★이 붙은 항목 설정
★ 데이터베이스 컨테이너 이름	WORDPRESS_DB_HOST	mysql000ex11
★ 데이터베이스 이름	WORDPRESS_DB_NAME	wordpress000db
★ 데이터베이스 사용자 이름	WORDPRESS_DB_USER	wordpress000kun
★ 데이터베이스 비밀번호	WORDPRESS_DB_PASSWORD	wkunpass

도커 컴포즈 정의 파일

- 컴포즈 파일 배치 경로

항목	값
컴포즈 파일 배치 경로(윈도우)	C:\Users\사용자명\Documents\com_folder
컴포즈 파일 배치 경로(macOS)	/Users/사용자명/Documents/com_folder
컴포즈 파일 배치 경로(리눅스)	/home/사용자명/com_folder

도커 컴포즈 정의 파일

- docker-compose.yml 파일 작성

- 1) 주항목 작성

```
version: "3"  
services:  
networks:  
volumes:
```


도커 컴포즈 정의 파일

- docker-compose.yml 파일 작성

2) 이름 작성

```
version: "3"
services:
  mysql000ex11:
  wordpress000ex12:
networks:
  wordpress000net1:
volumes:
  mysql000vol11:
  wordpress000vol12:
```

도커 컴포즈 정의 파일

- docker-compose.yml 파일 작성

3) MySQL 컨테이너의 정의 작성

```
version: "3"
services:
  mysql000ex11:
    image: mysql:5.7
    networks:
      - wordpress000net1
    volumes:
      - mysql000vol11:/var/lib/mysql
    restart: always
    environment:
      MYSQL_ROOT_PASSWORD: myrootpass
      MYSQL_DATABASE: wordpress000db
      MYSQL_USER: wordpress000kun
      MYSQL_PASSWORD: kunpass
  wordpress000ex12:
networks:
  wordpress000net1:
volumes:
  mysql000vol11:
  wordpress000vol12:
```

도커 컴포즈 정의 파일

- docker-compose.yml 파일 작성

4) 워드프레스 컨테이너의 정의 작성

```
version: "3"
services:
  mysql000ex11:
    image: mysql:5.7
    networks:
      - wordpress000net1
    volumes:
      - mysql000vol11:/var/lib/mysql
    restart: always
    environment:
      MYSQL_ROOT_PASSWORD: myrootpass
      MYSQL_DATABASE: wordpress000db
      MYSQL_USER: wordpress000kun
      MYSQL_PASSWORD: kunpass
```

5) 파일 저장

```
wordpress000ex12:
  depends_on:
    - mysql000ex11
  image: wordpress
  networks:
    - wordpress000net1
  volumes:
    - wordpress000vol12:/var/www/html
  ports:
    - 8085:80
  restart: always
  environment:
    WORDPRESS_DB_HOST: mysql000ex11
    WORDPRESS_DB_NAME: wordpress000db
    WORDPRESS_DB_USER: wordpress000kun
    WORDPRESS_DB_PASSWORD: wkunpass
networks:
  wordpress000net1:
volumes:
  mysql000vol11:
  wordpress000vol12:
```

도커 컴포즈

4절 - 도커 컴포즈 실행

도커 컴포즈 명령

- 컨테이너와 주변 환경을 생성하는 명령

```
docker-compose -f 정의_파일_이름 up 옵션
```

옵션 항목

옵션	내용
-d	백그라운드로 실행
--no-color	화면 출력 내용을 흑백으로 함
--no-deps	링크된 서비스를 실행하지 않음
--force-recreate	설정 또는 이미지가 변경되지 않더라도 컨테이너를 재생성
--no-create	컨테이너가 이미 존재할 경우 다시 생성하지 않음
--no-build	이미지가 없어도 이미지를 빌드하지 않음
--build	컨테이너를 실행하기 전에 이미지를 빌드
--abort-on-container-exit	컨테이너가 하나라도 종료되면 모든 컨테이너를 종료
-t, --timeout	컨테이너를 종료할 때의 타임아웃 설정. 기본은 10초.
--remove-orphans	컴포즈 파일에 정의되지 않은 서비스의 컨테이너를 삭제
--scale	컨테이너의 수를 변경

도커 컴포즈 명령

- 자주 사용하는 커맨드 예)

```
docker-compose -f C:\Users\사용자명\Documents\com_folder\docker-compose.yml up -d
```

- 현재 작업 디렉터리를 컴포즈용으로 사용할 때에는 따로 콤포즈 파일을 지정하지 않아도 됨

```
docker-compose up d
```

도커 컴포즈 명령

- 컨테이너와 네트워크 삭제 명령

```
docker-compose -f 정의_파일_이름 down 옵션
```

옵션 항목

옵션	내용
--rmi 종류	삭제 시에 이미지도 삭제한다. 종류를 all로 지정하면 사용했던 모든 이미지가 삭제된다. local로 지정하면 커스텀 태그가 없는 이미지만 삭제한다.
-v, --volumes	volumes 항목에 기재된 볼륨을 삭제한다. 단, external로 지정된 볼륨은 삭제되지 않는다.
--remove-orphans	컴포즈 파일에 정의되지 않은 서비스의 컨테이너도 삭제한다.

- 컨테이너를 종료하는 명령

```
docker-compose -f 정의_파일_이름 stop
```

도커 컴포즈 명령

- docker-compose 주요 명령 목록

커맨드	내용
up	컨테이너를 생성하고 실행한다.
down	컨테이너와 네트워크를 종료하고 삭제한다.
ps	컨테이너 목록을 출력한다.
config	컴포즈 파일을 확인하고 내용을 출력한다.
port	포트 설정 내용을 출력한다.
logs	컨테이너가 출력한 내용을 화면에 출력한다.
start	컨테이너를 시작한다.
stop	컨테이너를 종료한다.
kill	컨테이너를 강제 종료한다.
exec	명령어를 실행한다.
run	컨테이너를 실행한다.

create	컨테이너를 생성한다.
restart	컨테이너를 재실행한다.
pause	컨테이너를 일시 정지한다.
unpause	컨테이너의 일시 정지를 해제한다.
rm	종료된 컨테이너를 삭제한다.
build	컨테이너에 사용되는 이미지를 빌드 혹은 재빌드한다.
pull	컨테이너에 사용되는 이미지를 내려받는다.
scale	컨테이너의 수를 지정한다.
events	컨테이너로부터 실시간으로 이벤트를 수신한다.
help	도움말 화면을 출력한다.

도커 컴포즈 명령

- <실습> 도커 컴포즈 실행



- 실습에 사용할 컴포즈 파일

항목	값
윈도우	C:\Users\사용자명\Documents\com_folder\docker-compose.yml
macOS	/Users/사용자명/Documents/com_folder/docker-compose.yml
리눅스	/home/사용자명/com_folder/docker-compose.yml

- 컴포즈 파일 실행 및 확인

```
docker-compose up -d
```

```
$ docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS
PORTS
522eba9c9c6c   wordpress     "docker-entrypoint.s..." 53 seconds ago Up 49 s
0.0.0.0:8085->80/tcp   com_folder-wordpress000ex12-1
dd44395b2617   mysql:5.7     "docker-entrypoint.s..." About a minute ago Up 50 s
3306/tcp, 33060/tcp   com_folder-mysql000ex11-1
```

도커 컴포즈 명령

- 웹브라우저에서 워드프레스 접근
 - <http://localhost:8085>



- 컨테이너와 네트워크 종료 및 삭제

```
docker-compose down -d
```

```
$ docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS        NAMES
```